

Concurrency & Distributed Systems

Week 3: Deadlocks, Inspection, and Prevention

Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

Block 4 — Inspecting Deadlock

Goal

Move from theory to diagnosis.

Students should learn how to:

- identify BLOCKED threads
- identify lock ownership
- reconstruct circular wait from evidence

Engineering mindset

A deadlock is not only a concept. It is something you must be able to detect in a running system.

Inspecting Deadlock with jstack

```
jps  
jstack -l <PID>
```

Workflow

- ① Use `jps` to find the Java process ID
- ② Use `jstack -l` to dump all thread stacks
- ③ Search for blocked threads and monitor ownership
- ④ Reconstruct the waiting graph from the dump

Live-debugging mindset

Production is hung. CPU is low. No errors. You SSH in, run `jstack`, and start reading the system like evidence.

Normal Contention vs Deadlocked Dump

Normal contention

```
"Worker-1" ...  
- locked <0x1001>  
  
"Worker-2" ...  
- waiting to lock <0x1001>
```

Deadlock

```
"Worker-1" ...  
- locked <0x1001>  
- waiting to lock <0x2002>  
  
"Worker-2" ...  
- locked <0x2002>  
- waiting to lock <0x1001>
```

Key distinction

Two blocked threads on one lock may be healthy contention. Deadlock requires a **cycle**.

What Students Should Look For in a Thread Dump

"Thread-1" ...

- waiting to lock <0x00000007d5c8>
- locked <0x00000007d5b8>

"Thread-2" ...

- waiting to lock <0x00000007d5b8>
- locked <0x00000007d5c8>

Interpretation

Thread 1 holds one monitor and waits for the other. Thread 2 holds the other monitor and waits for the first.

State	Meaning	Deadlock relevance
BLOCKED	waiting for monitor entry	direct indicator
WAITING	waiting for notify/join	may be unrelated

How Monitor Addresses Reveal the Cycle

Key idea

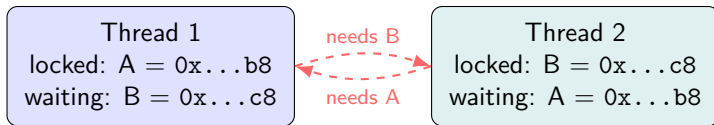
The hexadecimal monitor addresses are the evidence trail.

- `locked <0x...>` tells you what the thread owns
- `waiting to lock <0x...>` tells you what it needs
- Matching these addresses across threads reveals the dependency cycle

False-positive warning

Two threads blocked on the same lock do **not** automatically mean deadlock. Look for a closed cycle of ownership and waiting.

Thread Dump Reasoning Pattern



Pattern

Read the dump as a graph:

thread $\rightarrow$ holds lock $\rightarrow$ waits for other lock

If the graph closes back on itself, you have deadlock.

Mini-Challenge — Read the Dump

```
"Worker-A" ...  
- locked <0x1001>  
- waiting to lock <0x2002>
```

```
"Worker-B" ...  
- locked <0x2002>  
- waiting to lock <0x3003>
```

```
"Worker-C" ...  
- locked <0x3003>  
- waiting to lock <0x1001>
```

Question

Is this a deadlock? How many threads are involved? Draw the cycle.

Inspecting Deadlock with VisualVM

Visual cues matter

VisualVM lets students **see** the deadlock, not only read about it.

Use:

- the **Threads** tab
- the thread states view
- the deadlock detection feature

Psychological impact

When students see the tool reveal the bug automatically, deadlock stops being abstract and becomes real.

Placeholder for class version: insert annotated screenshot of VisualVM deadlock view.

Why Tooling Matters

- Theory explains why deadlock occurs
- Tools reveal where it occurs
- Thread dumps translate runtime behavior into analyzable evidence

Engineering lesson

A strong engineer does not stop at “I know deadlock.” A strong engineer can also diagnose it in a live or failing system.

Forward-looking note

Programmatic inspection is also possible with `ThreadMXBean.findDeadlockedThreads()`.

Programmatic Deadlock Detection Preview

```
ThreadMXBean bean = ManagementFactory.getThreadMXBean();  
long[] ids = bean.findDeadlockedThreads();  
  
if (ids != null) {  
    System.out.println("Deadlock detected");  
}
```

Why this matters

Manual inspection is essential for learning. Programmatic detection is essential for monitoring and observability.

Block 5 — Reducing Lock Risk

Goal

Not every lock causes deadlock, but poor lock design increases risk.

Practical strategies:

- ① Keep critical sections small
- ② Avoid nested locks when possible
- ③ Use global lock ordering
- ④ Avoid long blocking operations inside synchronized

Strategy 1 — Keep Critical Sections Small

Too broad

```
synchronized(account) {  
    complexCalculation();  
    Thread.sleep(100);  
    databaseCall();  
    account.update();  
}
```

Better

```
complexCalculation();  
databaseCall();  
  
synchronized(account) {  
    account.update();  
}
```

Rule of thumb

Lock the shared state, not the entire workflow.

Strategy 2 — Avoid Nested Locks When Possible

Meaning

Every additional lock increases structural risk.

- One lock can create contention
- Two or more locks can create dependency cycles
- Nested locking should be treated as a design decision, not a casual habit

Practical lesson

If one lock is enough, do not introduce two.

Strategy 3 — Use Global Lock Ordering

Meaning

If multiple locks must be acquired, all threads should acquire them in the same order.

- This prevents circular wait
- This is the cleanest structural deadlock-prevention strategy
- It scales from simple demos to real systems

Dynamic-resource teaser

If locks are chosen at runtime, order them by a stable rule: account ID, canonical key, or another immutable criterion.

Strategy 4 — Avoid Long Blocking Operations Inside synchronized

Examples of risky work inside a lock

- sleeping
- network I/O
- file I/O
- database calls
- waiting for user input

Why dangerous?

The longer a thread holds a lock, the longer other threads wait, and the higher the contention and deadlock risk.

Common Anti-Patterns That Increase Deadlock Risk

- calling external callbacks while holding a lock
- starting a thread inside a synchronized region
- performing long I/O under a monitor
- mixing multiple locks without a documented order

Engineering interpretation

Deadlock often appears not from one line, but from poor interaction between local design choices.

Lock Granularity Spectrum

Granularity	Advantage	Cost
Coarse-grained	simple reasoning	lower concurrency
Medium	balanced trade-off	moderate complexity
Fine-grained	higher concurrency	higher deadlock/comprehension risk

Lesson

More concurrency often means more reasoning complexity.

Performance vs Correctness

Important trade-off

Locks improve correctness, but excessive locking can reduce concurrency.

- Too little locking $\Rightarrow$ races and incorrect results
- Too much locking $\Rightarrow$ contention, blocking, and deadlock risk

Engineering balance

The goal is not “more locks.” The goal is **correct locking with minimal necessary blocking.**

Design Mindset for Safer Locking

- ① Ask what shared state actually needs protection
- ② Minimize the scope and duration of the lock
- ③ Check whether multiple locks are really necessary
- ④ Enforce a global order whenever multiple locks exist

Mindset

Do not treat synchronization as decoration. Treat it as a structural part of system design.

Week 3 Knowledge Check

- ① What is the difference between a race condition and a deadlock?
- ② If a thread is BLOCKED, is it always deadlocked?
- ③ Which Coffman condition does lock ordering eliminate?
- ④ Does synchronized help only with exclusion, or also with visibility?

Purpose

Before moving forward, students should connect safety, visibility, and liveness into one mental model.

End-of-Week Conceptual Summary

Students now understand:

- what a monitor is
- what mutual exclusion is
- what visibility means
- what happens-before guarantees
- how synchronization eliminates races
- how deadlock occurs
- how to prevent deadlock
- how to inspect blocked threads

What Changed This Week?

At the start

Students knew threads can run concurrently.

At the end

Students now understand that concurrency requires:

- control of shared-state access
- memory visibility guarantees
- liveness-aware lock design
- practical debugging of blocked systems

Key question

Is locking the only way to ensure correctness?

Preview:

- AtomicInteger
- hardware Compare-And-Swap (CAS)
- lock-free programming
- performance and contention

Concrete contrast

With locks, threads often queue and wait. With atomic operations, threads may retry without blocking.

Week 4 — Memory Visibility

Next week we go deeper into memory behavior itself:

- Java Memory Model
- `volatile`
- happens-before
- visibility vs atomicity

Recall prompt

`synchronized` guarantees visibility. What if we need visibility, but not full mutual exclusion?

Lab preview

- broken double-checked locking
- fixing it with `volatile`
- demonstrating stale reads

Week 4 Teaser — Why volatile Matters

```
if (instance == null) {  
    synchronized(lock) {  
        if (instance == null) {  
            instance = new Helper();  
        }  
    }  
}
```

Question

What could go wrong here without `volatile`? Think about visibility and instruction reordering.

Final Exercise for the Week

Take-home challenge

- 1 Write a small Java program that deliberately deadlocks
- 2 Use `jstack` to inspect it
- 3 Reconstruct the dependency cycle
- 4 Fix it with lock ordering
- 5 Verify that the deadlock disappears

Skill loop

Create → detect → explain → prevent

Final Thought for the Week

Big idea

Concurrency is not only about making things run at the same time.

It is about:

- controlling interference
- preserving correctness
- ensuring visibility
- maintaining progress

Mantra

Concurrency is not about stopping threads from running. It is about ensuring they run **correctly together**.